

Project Overview

My final term project is an interactive 3D implementation of the classic Tower of Hanoi puzzle built in Unity. The game supports one to eight disks, enforces all classical puzzle rules, tracks moves and elapsed time, displays the theoretical minimum number of moves, and includes an AI solver that demonstrates the optimal solution through animation. Players may interact with the disks and play manually, or they can let the AI solve the puzzle from scratch with the number of disks they selected. Players can also adjust the disk count at any time using an in-game slider.

Design and Implementation

My project is divided into five total scripts, where each one has a single and well defined responsibility.

My first script is ***Rod***: This file maintains a `Stack<GameObject>` representing the disks on each rod, handles the placement rule (no larger disk on a smaller one), world-space stacking position calculation, disk parenting, and collider management so only the top-most disk is ever clickable. For the world-space disk positioning, all disks are calculated in absolute world space using a fixed disk height constant. The formula used: $\text{worldY} = \text{DISK_HEIGHT} + \text{stackIndex} * (\text{DISK_HEIGHT} * 2)$ ensures correct stacking on any rod regardless of where that rod's parent `GameObject` sits in the scene. This decision was crucial as it eliminated bugs that arose from relying on local space coordinates, which were corrupted whenever a disk was re-parented between rods with different transforms.

My next script is ***DiskSpawner***: This file instantiates disk prefabs at game start and on reset. Uses `Mathf.Lerp` to distribute disk widths evenly between a minimum and maximum size regardless of disk count, ensuring disks always fit within the pole spacing. The file also holds an array of materials to color each disk a different color. I made four different materials (colors) for the disks, so once the number of disks is greater than four, the colors will cycle back and reset.

My next script is ***DiskMover***: This file handles player input. Casts a ray from the camera on each mouse click, resolves which rod was clicked via `GetComponentInParent<Rod>()`, and manages a two-click selection model: first click selects the top disk, second click attempts a move or cancels if the same rod is clicked again. If the player clicks a disk and attempts to place it on a rod, but the disk selected is larger than the top-most disk on the rod it wants to go to, the move will not execute (illegal move). However, the disk will still be selected to place somewhere (legally).

My next script is ***GameManager***: This file is the central coordinator. It tracks move count and elapsed time, updates all UI elements, processes the disk count slider, and orchestrates the reset and solve sequences.

My final script is ***TowerSolver***. This file implements the classic recursive Tower of Hanoi algorithm to generate the complete optimal move list, then executes those moves on the real board one at a time using a coroutine with a configurable delay between moves. Provides safety checks so that we do not violate Tower of Hanoi rules when solving the problem. The solver uses the classical recursive Tower of Hanoi algorithm, which produces the provably optimal solution of $(2^n) - 1$ moves for n disks. On solve, the game resets all disks to the starting rod, pauses briefly so the player can see the initial state, then executes each move with a configurable delay to create a watchable and followable animation. The move list is generated entirely before execution begins, keeping the planning logic cleanly separated from the animation logic.

Key Implementation Details

Hierarchy and Parenting Strategy: Each rod consists of a parent GameObject holding the ***Rod*** script and a child cylinder GameObject providing the visual mesh. ***Rod*** is intentionally placed on the parent, and not on the child. Disks are always parented to the rod parent, never to the child cylinder.

Collider Management: After every AddDisk or RemoveDisk operation, UpdateColliders() disables all disk colliders on the rod and re-enables only the top-most disk's collider. This enforces the rule that only the uppermost disk can be selected without requiring any additional logic in ***DiskMover***.

Lessons Learned

I have learned some great lessons doing this project. First, I saw first hand how modular design dramatically simplifies debugging and future modifications. By isolating responsibilities such as: move validation, UI updates, and algorithm execution - each component remained easy to reason about and test. Also, I learned that Unity's UI system requires deliberate configuration. Issues like text overflow, anchoring, and dynamic resizing highlights the importance of understanding RectTransforms and layout behavior. I had to understand these things to correctly implement the text panel that describes the overview of the game and the rules.

Future Work and Closing Remarks

I really enjoyed working on this project. I learned many valuable lessons that I hope to utilize in other projects moving forward. Originally, I wanted to do my term project on the classic Sudoku game. However, this seemed like a tall task given the time constraints. In the future I would like to be able to accomplish that project and maybe even a 3D chess game in Unity. This class has taught me so many interesting and useful things that I hope I can use in the future. From video game development introduction and event driven programming, all the way to game engine development, I have developed a great understanding of these core principles and it has undoubtedly made me a better video game designer. I did not know what to expect when enrolling in this course, but it was an amazing learning experience and I am glad that I did.

Project Screen Shots



